

Ex03 – Cryptographic Hash Functions (Resumo Explicativo e Simplificado)

1. O que é uma função hash criptográfica

Uma **função hash criptográfica** é um algoritmo que transforma dados de qualquer tamanho (texto, ficheiro, etc.) numa sequência de tamanho fixo chamada **digest** (ou resumo). O resultado é único para cada entrada, como uma impressão digital dos dados.

Exemplo: SHA256 gera sempre uma saída de 256 bits (64 caracteres hexadecimais), independentemente do tamanho do texto original.

2. Propriedades de uma boa função hash

- **1. Determinística:** o mesmo input gera sempre o mesmo output.
- **2. Avalanche effect:** uma pequena mudança no input causa uma grande diferença no output.
- **3. Irreversibilidade:** não é possível recuperar o texto original a partir do hash.
- **4. Uniformidade:** as saídas são distribuídas de forma aleatória.
- **5. Resistência a colisões:** é extremamente improvável que dois inputs diferentes gerem o mesmo hash.

3. Testar hashes no terminal

No Linux, podes usar os comandos abaixo para gerar hashes de um texto:

- **md5sum** — gera hash MD5 (128 bits).
- **sha256sum** — gera hash SHA-256 (256 bits).
- **sha384sum** — gera hash SHA-384 (384 bits).
- **sha512sum** — gera hash SHA-512 (512 bits).

Exemplo:

```
$ echo -n 'Seguranca Informatica' | sha256sum
```

O parâmetro **-n** evita que o comando *echo* adicione uma quebra de linha (`\n`) ao final do texto, garantindo que o hash seja calculado apenas sobre o texto desejado.

4. Avalanche Effect (Efeito Avalanche)

O **efeito avalanche** significa que uma pequena alteração na entrada — até mesmo um único bit — muda drasticamente o resultado do hash. Este comportamento é essencial para a segurança de uma função hash.

Para verificar isto, calcula-se o hash de duas mensagens que diferem num único caractere e compara-se o número de bits diferentes entre os dois hashes (chamado **Hamming Distance**).

Dica: caracteres '0' e '1' diferem em apenas um bit (ver tabela ASCII).

5. Análise estatística do efeito avalanche

O exercício pede para criar um programa (em C ou Python) que gere múltiplas mensagens aleatórias com apenas um bit alterado e calcule o número médio de bits diferentes entre os hashes. Isso serve para confirmar a distribuição aleatória das mudanças no hash.

- **Ferramentas:** OpenSSL em C ou módulo cryptography em Python.
- **Comando C:** gcc avalanche-analysis.c -o avalanche-analysis -lcrypto
- **Comando Python:** sudo apt install python3-cryptography
- **Visualização:** usar gnuplot para criar um gráfico de distribuição.

6. Proof-of-Work (Prova de Trabalho)

A prova de trabalho é um mecanismo que demonstra que uma tarefa custosa em termos de tempo/computação foi realizada. Ela é usada em sistemas como o Bitcoin para validar blocos.

No exercício, o objetivo é encontrar uma frase que, ao ser passada por uma função hash (como SHA-256), gere um resultado que comece com um certo número de zeros (ex: três zeros em hexadecimal).

Exemplo:

```
$ echo -n 'My student number is 1234 and I love crypto!' | sha256sum
```

Para encontrar um hash com três zeros iniciais, é preciso tentar várias combinações — o que representa o trabalho computacional exigido.

7. Aplicações das funções hash

- Verificação de integridade de ficheiros (md5sum, sha256sum).
- Armazenamento seguro de senhas (hash + salt).
- Assinaturas digitais e certificados SSL.
- Provas de trabalho em blockchains.

8. Perguntas possíveis (e respostas curtas)

- ■ O que é uma função hash criptográfica?
→■ Um algoritmo que transforma dados em um resumo único e irreversível.
- ■ O que é o efeito avalanche?
→■ Uma pequena mudança no input altera totalmente o hash.
- ■ O que é a Hamming Distance?
→■ Número de bits diferentes entre dois hashes.
- ■ Qual o tamanho da saída do SHA-256?
→■ 256 bits (64 caracteres hexadecimais).
- ■ O que é uma colisão?
→■ Quando dois inputs diferentes produzem o mesmo hash.
- ■ Para que serve a prova de trabalho?
→■ Mostrar que foi realizado um cálculo dispendioso (ex: mineração).
- ■ O que faz o comando echo -n?
→■ Evita adicionar uma quebra de linha no texto antes de gerar o hash.